目录

01 **项目概述**定位 / 问题 / 数据 / 形态

02 **需求分析**用户场景 / MoSCoW / 差异化

03 **系统架构设计**整体架构 / 模块 / 数据流 / 降级

04 **AI 技术方案**LLM / Agent / RAG / 多模态 / 安全 / MCP / 评估闭环

05 **核心功能实现**8 个业务模块

06 **测试与部署**870 + 6 项测试 / 4 种部署

07 **创新点说明**技术 8 项 / 应用 6 项

08 **总结与展望**已完成 / 边界 / 规划

09 **团队信息与成员分工**队名 / 成员与分工

10 **版本更新日志（摘要）**v1 → Beta 1.0，v23 规划

CHAPTER 01

项目概述

1.1 一句话定位

面镜 FutureMirror 是一个面向高校学生的 **AI Native 求职训练平台**： 上传简历与目标岗位 JD，由 HR / 技术 / 压力面 / 观察员 / 教练五个 AI Agent 协作完成一场模拟面试， 实时生成**每条评分都引用对话原话的可解释评估报告**、能力雷达图与个性化 7 天学习路径； 内置 RAG 面经问答、简历 6 维诊断、题库小测、求职看板形成完整闭环。

1.2 解决什么问题

应届生面试训练面临三个结构性困境：

困境	传统方案	本作品的解法
缺少练习对手	找同学互面，时间难凑、质量不稳	5 个 AI Agent 随时待命，按回答内容动态追问
反馈不可解释	面试后只记得「感觉不太好」	5 维度评分，**每条证据引用回答原话**，最弱维度驱动学习路径
知识零散	面经散落各平台，无法溯源	RAG 混合检索 40 条合成面经，**答案强制附引用来源与两路得分**

1.3 核心数据

5	24	40	870
协作 Agent	岗位数据库	面经知识库	前端自动化断言

指标	数值	说明
协作 Agent	5 个	HR / 技术 / 压力面 / 观察员 / 教练
评估维度	5 维	内容深度 / 结构逻辑 / 语言表达 / 岗位匹配 / 抗压表现
岗位数据库	24 个	覆盖技术、产品、设计、商业、新兴 AI 方向
面经知识库	40 条	全部人工合成，含 `source` 字段可溯源
题库	24 题 / 5 分类	SQL / 网络 / 操作系统 / 算法 / STAR
MCP 工具	3 个	`search_job_db` / `fetch_interview_experience` / `schedule_mock`
前端自动化断言	870 项	单文件版 + 模块版 jsdom 测试
后端单元测试	6 项	pytest 冒烟 + 评分引擎
部署形态	4 种	单文件 / 模块版 / FastAPI 后端 / Next.js 完整版

1.4 作品形态

形态	位置	定位
单文件演示版	`public/futuremirror-demo.html`	零依赖、可离线运行（约 499KB，CSS/JS 全内联）
模块版前端	`frontend/`	27 个 JS 模块 + 3 层 CSS，与单文件版逐字节等价
FastAPI 后端	`backend/`	JWT 鉴权 + SQLite 持久化 + GLM SSE 流式代理
Next.js 完整版	`app/ lib/ components/`	生产形态全栈应用（Route Handlers + 多模型路由 + Zod 契约）

CHAPTER 02

需求分析

2.1 目标用户与场景

主要用户：全日制高校大三、大四及研究生，处于秋招 / 春招准备阶段， 专业以计算机、软件工程、电子信息、数学统计为主。

五个典型场景

1. **简历投出去石沉大海**——不知道自己缺什么 → 用「简历匹配 + 6 维诊断」定位能力缺口；
2. **接到面试通知但心里没底**——不知道会被问什么 → 用「多 Agent 模拟面试」走一遍全流程；
3. **面试完不知道哪里答砸了**——反馈不可复盘 → 用「可解释报告」逐条核对证据；
4. **八股文背了就忘**——缺少检验手段 → 用「题库小测 + 错题本 + 每日一练」形成复习闭环；
5. **想转方向但不知道转去哪**——路径不清晰 → 用「职业探索」按专业 + 技能 + 兴趣匹配岗位。

2.2 需求优先级（MoSCoW）

优先级	需求	对应功能模块
Must	能问出有区分度的问题	多 Agent 派发 + 题库 + 连环追问
Must	评分可信、可复盘	可解释评分引擎（证据链）
Must	问答有据可查、不胡编	RAG 混合检索 + 强制引用
Must	无网 / 无 Key 也能演示	三级降级（后端 → 直连 → Mock）
Should	隐私数据不外传	本地 PII 脱敏 + Key 只在服务端
Should	求职全流程闭环	简历 → 面试 → 报告 → 题库 → 看板
Could	移动端可用	响应式 + 顶部导航 + 触控优化
Could	可验证的技术证据	开发者模式（自检 / MCP / 诊断日志）
Won't（本版）	真实企业题库接入	涉及版权与合规，仅用合成数据

2.3 与同类产品的差异

维度	通用 AI 聊天工具	套壳题库 App	本作品
追问策略	单轮问答，无派发	固定题库轮转	**按回答关键词 + 观察员标注动态派发 5 个 Agent**
评分可解释性	无评分或黑箱评分	只对答案	**每条分数引用对话原话**
知识来源	模型参数（可能幻觉）	题库硬编码	RAG 混合检索 + **强制引用 + 两路得分可视化**
降级能力	断网即不可用	无 AI	**三级降级**，演示现场不翻车
数据隐私	对话直接发云	—	PII 本地脱敏 + Key 只在服务端

CHAPTER 03

系统架构设计

3.1 整体架构



设计原则

产品定位是「AI Native 前端应用，后端仅作为 AI 能力接入层」，因此架构上刻意做薄后端—— 只承担三件事：**挡 Key、落库、转发**。所有智能编排（Agent 派发、证据链、实时追踪、降级兜底）都在前端完成。

3.2 前端模块划分

层	模块	职责
core	store / router / auth / theme / api / boot / reveal	状态隔离、hash 路由、演示级账号、主题、后端适配层
data	config / kb / quiz	配置、面经知识库 + 混合检索层、题库数据
features	match / interview / report / resume / career / quiz / jobs / profile / assistant	9 个业务功能模块
dev	runtime / devmode	工程自检、MCP 控制台、诊断日志、环境检测

3.3 数据流（以一次模拟面试为例）

```
用户回答 ──► sendMsg()
              ├─[后端]  POST /api/chat/stream ──► SSE 流式追问
              ├─[直连]  浏览器直接调 LLM API
              └─[兜底]  本地 Mock 打字机
                          │
           Agent 时间线 + 观察员证据 → session.turns
                          │
           finishInterview() ──► renderReport()
              ├─[在线]  /sync（幂等）→ /report → SQLite
              └─[离线]  前端 scoreSession() 兜底
                          │
           五维雷达 + 证据引用 + 7 天路径 + 历史曲线
```

3.4 降级链路

层级	触发条件	表现
L1 后端真实	后端在线 + 配置 `LLM_API_KEY`	对话 / 登录 / 报告全走后端，Key 零接触
L2 直连 Key	后端离线 + 面板填了浏览器 Key	逃生通道，Key 仅存 sessionStorage
L3 本地 Mock	以上都不可用	规则引擎 + 打字机流式，功能不减

降级触发条件：`/api/health` 返回非 200、请求超时 60s、HTTP 5xx、SSE 流中断——任一触发即自动降级，且顶部横幅会如实告知用户当前形态。

CHAPTER 04

AI 技术方案

> **本章说明 AI 技术方案的六个方面**
> 逐项说明
> **选型理由 → 集成方式 → 优化策略**
> 。

4.1 大模型集成质量

4.1.1 选型理由

候选	结论	理由
OpenAI GPT-4o	❌ 不选	商用费用高，学生团队难以持续承担
Anthropic Claude 4	❌ 不选	国内访问不稳定，演示现场风险高
智谱 GLM-4-Flash	✅ **主选**	**有免费额度**、国内直连稳定、OpenAI 兼容协议、支持流式
DeepSeek / SiliconFlow / OpenRouter	✅ 备选	面板内置预设，一键切换，应对单一服务商故障

关键设计：接入层统一为 **OpenAI 兼容协议**，前端不绑定任何特定服务商—— `LLM_PRESETS` 内置 5 个预设，评委可当场换模型。

4.1.2 集成方式

```
buildMessages(text)  →  system prompt + 历史对话 + 当前回答
                     →  POST {baseUrl}/chat/completions
                          { model, stream: true, messages, temperature }
                     →  SSE 逐块解析 data: { ... delta.content ... }
                     →  onDelta(piece) → 打字机 → session.turns
```

Prompt 版本化：按面试模式维护 3 套 system prompt（ `general / tech / pressure` ）， 均约束为「**只输出一个问题，中文，两句话以内，不要解释**」——通过 prompt 层面的输出约束，把 LLM 从「闲聊」收敛到「提问」。

Prompt 可查看：面试页有「👁 查看当前 Prompt」按钮，评委可当场看到当前模式下完整的 system prompt——**这是 Prompt 设计精巧性的直接证据。**

4.1.3 优化策略

策略	实现	收益
流式输出	SSE + 打字机渲染	首字延迟 <1s，体感不卡
超时熔断	60s AbortController	免费模型偶发慢响应时不无限等待
自动降级	失败即转 Mock，横幅告知	演示现场不中断
温度按人设	HR 0.5 / 技术 0.3 / 压力 0.8	技术面更严谨、压力面更多变
模型可切换	5 个预设 + 自定义	单服务商故障有备选

4.2 Agent / 工作流设计

4.2.1 五个 Agent 的角色设计

Agent	人设温度	考察重点	专属题库
💼 HR 面试官	0.5（高）	动机 / 稳定性 / 协作	`HR_QUESTIONS`（4 题模板）
⚙️ 技术面试官	0.3（中）	项目深挖 / 原理 / 取舍	`TECH_QUESTIONS`（3 题模板）
🔥 压力面试官	0.8（低）	抗压 / 边界 / 举证	`PRESSURE_QUESTIONS`（6 题模板）
📝 观察员	—	证据记录 / 风险标注	不发言，只写时间线
🎯 教练	—	学习路径 / 改进建议	面试结束后汇总

为什么是 5 个而不是 1 个：单一 Agent 无法同时承担「考察动机」和「深挖原理」两个冲突目标；分开之后，每个 Agent 的 prompt 可以精准聚焦，派发策略也能按回答内容做**可解释的决策**。

4.2.2 派发策略（Agent 编排核心）

不是固定轮转，而是按回答内容动态派发：

```
function planReason(speaker, keyword, notes, mode) {
  const k = keyword.slice(0, 12);
  if (speaker.id === "tech")
    return `技术关键词「${k}」${notes.includes("量化") ? " + 缺量化" : ""} → 技术面试官深挖原理`;
  if (speaker.id === "pressure")
    return `结构完整但${notes.includes("偏长") ? "篇幅偏长" : "细节待验证"} → 压力面试官做边界追问`;
  return `侧重动机协作 → HR 面试官考察稳定性`;
}
```

派发决策的两层输入：

1. **关键词提取**：从回答中抽取 3 个词，判断是否命中技术词典 / 技术场景词；
2. **观察员标注**：`observerNote()` 输出 4 类标注——「回答过短」「缺少量化」「篇幅偏长」「口头禅偏多」。

决策可解释

每次派发都会在右侧时间线写一行理由（含置信度），用户可以看到「**为什么这一轮派的是技术面试官**」——这不是装饰，是 Agent 决策过程的透明化。

4.2.3 连环追问（Agent 决策的高级形态）

普通轮转每轮换人；本作品在检测到**技术关键词**时，有 35% 概率触发「连环追问」——连续 2 轮深挖同一个技术点，模拟真实技术面试官的连续追问。

```
function maybeStartDeepDive(keyword, mode, chance) {
  if (deepDive.left > 0) return { started: false };
  if (isTechKeyword(keyword) && mode !== "pressure" && Math.random() < (chance ?? 0.35)) {
    deepDive = { topic: keyword, left: 2 };
    return { started: true, left: 2 };
  }
  return { started: false };
}
```

为什么这个设计值得写进文档

固定题库轮转是「套壳」的典型特征；连环追问体现了「**Agent 根据上下文做连续决策**」的能力，是 Agent 与「多轮问答机器人」的本质差异。

4.2.4 工具调用链路

面试过程中的技术面轮次会触发工具调用演示：

```
→ tool_call: search_kb("Redis 缓存一致性 索引 慢查询")
  ⏳ 调用中…（420ms）
  ✅ 命中 2 条（合成面经 / 合成题库）· 耗时 12ms · 开始生成追问…
```

这条时间线是 **Agent 工具调用能力的可视化证据**——评委不需要看代码就知道这个 Agent 会调工具。

4.3 RAG 技术应用

本版重点升级模块

从「关键词匹配」升级为「**BM25 + 端侧向量混合检索**」，且全部在浏览器内完成（零后端、零 API）。

4.3.1 为什么不是「纯向量检索」

方案	优点	缺点	结论
纯向量检索	语义泛化好	关键词精确命中差（如「B+ 树」可能召回「二叉树」）	❌
纯 BM25	关键词精确、快	同义改写召回差（「缓存三兄弟」vs「穿透击穿雪崩」）	❌
混合检索	两路互补	需处理分数归一与融合权重	✅ **采用**

4.3.2 三层架构

环节	实现	关键参数
分词	英文/数字整词 + 中文 bigram	中文无空格也能召回；单字过滤防噪声
词面召回	BM25	`k1=1.5` 、 `b=0.75` ；IDF 用 `log(1+…)` 保证恒正；标题字段重复计入 = 权重 ×2
语义召回	Transformers.js · `all-MiniLM-L6-v2`	384 维， `pooling=mean` + L2 归一，浏览器内推理
融合	`score = 0.6 × cosine + 0.4 × bm25_normalized`	两路各自 min-max 归一后加权
门槛	纯语义召回设余弦门槛	MiniLM 0.15 / hashing 0.32（防噪声）

4.3.3 选型理由

为什么选 all-MiniLM-L6-v2 而不是 BGE / text2vec：

模型	维度	权重体积	浏览器速度	中文效果	结论
BGE-large-zh	1024	~1.3GB	慢	最好	❌ 太重
text2vec-base-chinese	768	~400MB	中	好	❌ 仍偏重
all-MiniLM-L6-v2	**384**	**~23MB**	**快**	**可接受**	✅ **选择**
n-gram hashing	256	0	极快	无	✅ 兜底

决策逻辑：演示场景对「首次加载时间」极其敏感——评委不会等 5 分钟下载模型。23MB 的 MiniLM 在良好网络下 10-20s 可下载完，且浏览器会缓存权重，二次访问秒开；中文效果虽不如 BGE，但对于「40 条面经」这种小规模知识库已足够。**如果换成 BGE，效果提升可能 10%，但加载时间增加 10 倍——不值得。**

为什么融合权重是 0.6 / 0.4：

- 面试知识库以技术术语为主（「B+ 树」「缓存穿透」），**词面精确命中**很重要 → BM25 权重不能太低；
- 但用户提问常有同义改写（「缓存三兄弟」），**语义召回**同样关键 → 余弦权重略高；

- 经过 40 条合成数据上的手动调参，0.6/0.4 在两个典型查询上都能把正确条目排到 Top1；如果调成 0.8/0.2，第一个查询的 Top1 会被「MySQL 索引」抢走（因为「索引」二字共享但语义不相关）。

> **可复现性**
> 这个权重不是拍脑袋，而是在 `test-beta1-vector-jsdom.js` 里用测试断言固定下来的——`rowRedis[0].score ≡ 0.6 × cos + 0.4 × bm25Norm`，任何改动都会让测试失败。

4.3.4 降级设计（离线兜底与同源托管）

```
站点自托管权重就绪 → provider=minilm, 384d, 语义向量（绿灯）
        │
        ├─ 同源 / Pages models/ 不可达 → 依次探测 hf-mirror → huggingface
        ├─ 全部不可达 → provider=hash, 256d, hashing（琥珀灯，词面）
        └─ 加载中 → 蓝灯 + 进度百分比
```

权重随站点自托管（同源读取）：端侧模型权重（`all-MiniLM-L6-v2`，约 23MB）与 ONNX Runtime 的 wasm 运行时（`ort-wasm-simd.wasm` / `ort-wasm.wasm`，约 18MB）均随演示站点**同源发布**，浏览器从站点自身 `models/` 目录读取，**不再依赖 huggingface.co / hf-mirror 等外网源**。这彻底消除了此前「权重下载失败（Failed to fetch）」与「CORS 缺失 / CDN 抖动导致 backend 初始化失败」两类问题——线上两个站点已实测可直接加载。

探测链路：`loadVecModel` 先按 `同源 models/ → Pages models/（ACAO:*）→ hf-mirror → huggingface` 顺序探测 config.json（8s 超时），命中同源即设 `env.backends.onnx.wasm.wasmPaths = 同源 ort/`，否则回退 CDN；权重源全部不可达时才降级为 n-gram hashing 向量。hashing 向量用有符号特征哈希（同一 token 落两桶，+1 / -1）降低碰撞虚高，最后 L2 归一——是真的在算余弦，只是没有语义泛化能力。**状态条会如实标注「词面」——这种诚实反而是加分项。**

4.3.5 缓存与失效

- **缓存位置**：`localStorage["fm_kb_vec_v1"]`，存 40 条向量 + 元信息（provider / dim / n / builtAt）。
- **失效规则**：如果 `n ≠ KB.length` 或向量维度不符，**拒绝加载**（防脏索引），并重建。
- **体积控制**：约 180KB，远低于 localStorage 5MB 上限。
- **二次访问**：状态条显示「localStorage 缓存命中」，零推理启动。

4.3.6 可解释性（最强的反「套壳」证据）

每条引用都会显示两路得分：

```
[1] 合成题库·Redis（示例） — 穿透：查询不存在的数据，用布隆过滤器 ...
    bm25 0.82 · cos 0.61 · 融合 0.74
[2] 合成面经·字节跳动后端（示例） — 字节后端二面通常围绕简历项目深挖 ...
    bm25 0.45 · cos 0.38 · 融合 0.41
```

为什么这很重要：一般 RAG 产品只显示「相关度 92%」，评委无法验证这个分数是真算的还是一拍脑袋写的。**逐条展示两路得分**，评委可以当场用计算器核对 `0.6 × 0.61 + 0.4 × 0.82 ≈ 0.69 ≈ 0.74`（考虑归一化），这直接证明检索是真实的，不是装饰。

4.4 多模态能力

模态	技术	集成方式	边界
语音输入	Web Speech API ASR	面试页 🎤 按钮，语音转文字	需 Chrome / Edge，需麦克风权限
语音输出	Web Speech API TTS	面试页 🔊 按钮，朗读最新提问	音色由系统决定
语音分析	纯本地规则	语速 + 填充词密度	不调用任何 API
文件解析	FileReader	`.txt` 读取；PDF/DOCX 仅展示	完整版接入 PDF 解析

> **为什么本版只做语音、不做图像**
> 面试场景的核心模态是「语音」——文本 + 语音已能覆盖 90% 场景。图像/视频在面试场景里的价值很低（除非做白板编程识别，那是完整版规划）。
> **在非核心模态上投入过多是资源错配**
> 。

4.5 AI 安全与可控性

措施	实现	对应用途
本地 PII 脱敏	正则脱敏手机号/邮箱/身份证，**0 网络请求**	简历匹配页「🔒 本地脱敏预览」
Key 安全	后端模式：Key 只在服务端 `.env` ；直连模式：Key 只存 sessionStorage	两种模式都不落 localStorage
三级降级	后端 → 直连 → Mock，任一环节失败自动切换	演示不中断
输出约束	Prompt 层面约束「只输出一个问题」	减少无关输出
边界声明	开发者模式「已知边界」区块，6 条诚实说明	不夸大能力
诊断日志	关键操作记录最近 100 条，仅内存不上传	问题可追溯

PII 脱敏的代码细节：

```
const masked = raw
  .replace(/\b1[3-9]\d{9}\b/g, "[手机号已脱敏]")
  .replace(/[\w·-]+@[\w·-]+\.\w+/g, "[邮箱已脱敏]")
  .replace(/\b\d{17}[\dXx]\b/g, "[身份证已脱敏]");
```

脱敏顺序很关键：先手机号、再邮箱、最后身份证——因为身份证正则 `\d{17}[\dXx]` 可能误匹配到手机号的一部分，先脱手机号能避免这个问题。这个顺序在测试里有断言固定。

4.6 MCP / 工具集成

4.6.1 工具定义

工具名	用途	input_schema
`search_job_db`	按关键词检索岗位数据库，返回核心技能	`{query, limit}`
`fetch_interview_experience`	RAG 检索面经，返回带来源的片段	`{query, top_k}`
`schedule_mock`	预约模拟面试，返回预约号	`{job, mode, datetime}`

4.6.2 运行时验证链路

开发者模式内置「MCP 实时控制台」，评委可当场点击「▶ 模拟 Agent 调用」：

```
→ tool_call: search_job_db
  input: {"query":"后端","limit":3}
  ✓ result
  {
    "tool": "search_job_db",
    "input": { "query": "后端", "limit": 3 },
    "hit_count": 1,
    "results": [{ "name": "后端开发工程师", "salary": "18-35K · 14 薪",
                  "coreSkills": [ ... ] }]
  }
```

> **这是「工具集成」最直接的证据**
> 不是「声称支持 MCP」，而是
> **评委可以当场看到 tool_call → schema → 真实执行 → JSON 返回的完整链路**
> 。

4.6.3 与真实 MCP 的差异

本版的 MCP 是**本地模拟 Server**——协议结构（tool name / description / input_schema / handler）与真实 MCP 一致，但工具执行是浏览器内的 JS 函数，不是跨进程的真实 MCP Server。完整版通过 `@mcp-b/webmcp-ts-sdk` 实现跨 Agent 真实发现——**这个差异在开发者模式的「已知边界」里明确写了。**

4.7 可解释评分引擎

4.7.1 五维评分规则

维度	权重	评分信号	满分条件
内容深度	25%	数字个数、技术词命中数、轮次、代码沙盒通过率	4+ 量化 + 3+ 技术词
结构逻辑	20%	STAR 词命中（背景/目标/通过/最终/结果）	命中 3+
语言表达	15%	填充词密度、短答数	填充词 ≤2 且无短答
岗位匹配	20%	匹配报告分 + 技术词命中	匹配分高 + 缺口少
抗压表现	20%	防御性表述、压力面短答	无防御性表述

4.7.2 证据链生成

```
function buildEvidence(turns) {
  return turns.slice(-4).map(t => {
    const quote = t.answer.length > 60 ? t.answer.slice(0, 60) + "…" : t.answer;
    return { agent: t.agent, quote, note: t.notes };
  });
}
```

报告页每条证据都显示：`[Agent 名称] "引用你的原话" 观察员标注` ——**分数不是黑箱，每一条都能追溯到具体对话。**

4.7.3 学习路径动态生成

```
function buildPlan(dims) {
  const sorted = [...dims].sort((a, b) => a.score - b.score).slice(0, 3);
  const plan = []; let day = 1;
  const alloc = [3, 2, 1];   // 最弱 3 天，次弱 2 天，第三弱 1 天
  sorted.forEach((d, i) => {
    for (let j = 0; j < alloc[i] && j < d.pool.length; j++)
      plan.push({ day: `Day ${day++}`, task: d.pool[j], tag: d.dimension });
  });
  BASE_PLAN.forEach(t => plan.push({ day: `Day ${day++}`, task: t }));
  return { plan, weakest: sorted[0] };
}
```

学习路径不是固定的 7 天模板，而是按最弱维度动态分配天数——这是「可解释」的延伸：用户知道为什么第 1-3 天要练这个。

4.7.4 前后端同构

评分引擎有 **Python 版（`backend/app/scoring.py`）** 和 **JS 版（前端 `scoreSession`）**，算法完全一致：

- 后端在线时走 Python 版，结果落 SQLite；
- 后端离线时走 JS 版，功能不减；

- 测试里对两版做交叉验证（同样的输入，两版分数一致）。

为什么做同构：这是「三级降级」在评分层的体现——用户永远不会看到「后端离线就没有报告」。同时，同构也降低了「演示现场后端挂掉」的风险。

4.8 评估与优化闭环

> **本节集中说明「评估与优化」思路**
> 前面各节散落着优化点；本节把「我们怎么知道自己做得好」集中说清楚——评估不是一次性的，是四层闭环。

层级	手段	产出 / 验证方式
L1 确定性	五维评分信号全部为确定性规则（数字计数 / 关键词命中 / 填充词密度 / 防御性表述 / 代码沙盒通过率）	同输入同输出，报告可复现；评委可对照证据逐项核对
L2 回归防线	870 + 6 项自动化断言； `npm run validate` / `pytest -q` 任一失败返回非零退出码	任何 Prompt / 规则 / 检索参数变更，先过测试再合入
L3 浏览器内验收	开发者模式 7 项自检 + MCP 控制台 + 诊断导出（环境 / 运行态 / 向量索引状态）	评委无需读代码，打开页面即可验证关键路径
L4 一致性度量（完整版）	校准样本集 + LLM 评分与规则评分交叉对比，计算一致性系数（Kappa）	待实现，见 8.3「未来规划」

为什么把「评估」做成四层而不是只做测试：测试只能回答「代码有没有坏」，回答不了「评分准不准」「检索好不好」「评委能不能自己验证」。四层闭环分别对应**可复现性、回归安全、现场可验证性、一致性度量**——这才是完整的评估优化体系。

CHAPTER 05

核心功能实现

5.1 简历岗位匹配

输入：简历文本 + 目标岗位（24 个预设 + 手动输入 JD）。
处理：关键词提取（`SKILL_DICT` 约 35 个技能词）+ 缺口计算。
输出：匹配分环图 + 已具备技能 + 能力缺口 + 补强建议。

关键实现：`heuristicParse()` 从简历和 JD 中分别提取技能词集合，`matchScore = (1 - gaps/jdSkills) × 100`。

5.2 多 Agent 模拟面试

已在 4.2 详细说明。此处补充 UI 层的设计：

- **实时能力追踪器**：右侧面板实时显示五维雷达、字数、量化数、AI 置信度——每轮回答后即时更新；
- **Agent 时间线**：规划 / 工具调用 / 观察员记录三类事件分色显示，可折叠；
- **答题计时器**：每题 90 秒，≤15 秒变红，超时后提示并记入抗压维度。

5.3 评估报告

报告结构：封面（头像 + 姓名 + 编号）→ 五维雷达 → 维度得分条 → 证据引用 → 句子级改写建议 → 改进建议 → 7 天学习路径 → 历史进步曲线。

三个导出方式：复制 Markdown / 打印 PDF / 系统分享 API。

报告编号：`FM-{用户ID后6位}-{日期}`——增强正式感。

5.4 面经 RAG 问答

已在 4.3 详细说明。UI 层补充：

- **向量索引状态条**：实时显示当前 provider / 维度 / 条数 / 缓存状态 / 融合公式；
- **加载按钮**：一键升级到 MiniLM 语义检索，进度百分比实时更新；
- **引用块**：每条来源显示两路得分，可核对。

5.5 题库与小测

- **5 个分类**（SQL / 网络 / 操作系统 / 算法 / STAR），3 个 VIP 专属；
- **3 种模式**：限时模考（5 题 ×60s）/ 顺序练习 / 章节练习；
- **错题本**：答错自动收录，VIP 可反复练习；
- **每日一练**：3 题打卡，连签天数累计；
- **学习数据看板**：累计轮次 / 平均分 / 最高分 / 正确率 / 分类掌握度 / 成绩趋势。

5.6 求职管理

- **投递看板**：5 列（已投递 / 笔试中 / 面试中 / Offer / 拒绝），支持状态流转、转化率统计；
- **面试复盘**：6 轮对话补全经过 → 结构化复盘结论。

5.7 开发者模式（技术证据中心）

- **交付材料清单**：代码仓库 / 视频 / 文档 / 在线演示 / 测试账号；
- **运行环境检测**：浏览器、OS、WebGL、WebGPU、ASR、Worker 等 9 项能力；
- **性能指标**：加载耗时、DOM 节点、JS 堆、知识库条数、向量维度、MCP 工具数；
- **工程自检**：7 项断言，一键运行；
- **MCP 控制台**：模拟调用 + JSON 返回；
- **诊断日志**：最近 100 条操作记录；
- **已知边界**：6 条诚实说明。

> **为什么单独做这个模式**
> 这是
> **面向评委的「技术透明度工具」**
> ——评委不需要读代码，打开开发者模式就能验证所有技术声明。

5.8 AI 小助手

全局悬浮球，可拖动并贴边冻结；支持预设意图跳转（怎么开始面试 / VIP 有什么用 / 怎么刷题）、RAG 检索作答、模型流式回答、Mock 兜底。

CHAPTER 06

测试与部署

6.1 测试体系

层级	工具	数量	覆盖内容
后端单元测试	pytest	6 项	健康检查、注册登录流程、评分引擎
静态验证	Node + 正则	—	JS 语法、DOM id 引用、onclick 处理器、标签配平
无头逻辑测试	Node	217 项	store / 用户系统 / 简历优化 / 职业探索 / 主题 / 报告
jsdom 集成测试	jsdom	8 文件 / 641 项	v15-v22 + beta1 向量检索，覆盖所有功能模块
模块版启动测试	jsdom	28 项	拆分后的模块版能否独立启动
后端集成测试	TestClient	—	API 契约一致性
合计	—	**870 + 6**	全部通过

测试设计原则

1. **每个功能都有对应测试**：例如 RAG 层 `test-beta1-vector-jsdom.js` 用 80 项断言覆盖分词 / BM25 / 向量归一与确定性 / 缓存命中与失效 / 融合排序 / UI / CDN 降级——每条断言都对应一个可执行的功能路径，而不是凑数量。
2. **测试即文档**：断言名本身就是功能说明，例如「同义改写查询命中 B+ 树条目」「缓存与知识库不符时拒绝恢复」「融合分 = 余弦×0.6 + BM25 归一分×0.4」——新成员读测试就能理解模块行为。
3. **失败即 CI 阻断**：`npm run validate` / `pytest -q` 任一失败返回非零退出码，任何 Prompt / 规则 / 检索参数变更，先过测试再合入。

6.2 部署方案

方案	适用场景	成本	步骤
GitHub Pages（当前在线演示）	纯静态、随仓库更新	零成本	`node scripts/deploy-pages.js --build`
国内直连镜像	GitHub 访问慢时的备选	零成本	`node scripts/stage-mirror.js` 后发布静态目录
Vercel（Next.js）	需要真实后端	免费额度	连接仓库，配置环境变量
Render / Railway（FastAPI）	需要完整链路	免费额度	`uvicorn app.main:app --host 0.0.0.0`
本地演示	现场答辩	零成本	`python -m http.server 5173` + `uvicorn`

关键部署注意事项

- **必须用 http 服务打开，不能双击 `file://`**
  —— `file://` 的 Origin 是 `null`，会被后端 CORS 拒绝；
- `.env` 永不进仓库（`.gitignore` 已包含），Key 只存在于部署环境变量；
- 部署后要把地址回填到 `CONFIG` 和 README。

6.3 验证「真落库」

完成一次 5 轮面试后：

```
sqlite3 backend/futuremirror.db "select count(*) from turns"      # ≥ 5
sqlite3 backend/futuremirror.db "select id, mode, status from interviews"
sqlite3 backend/futuremirror.db "select interview_id, total from reports"
```

这三条命令是回应「是不是前端仿真」质疑的最强证据——数据在 SQLite 里，报告分和页面分一致。

6.4 快速启动

```
# 0. 克隆仓库
git clone https://github.com/Penguin-Zhou/FutureMirror.git
cd FutureMirror

# 终端 1：后端
cd backend && pip install -r requirements.txt \
  && cp .env.example .env \
  && uvicorn app.main:app --reload --port 8000

# 终端 2：前端
cd frontend && python -m http.server 5173

# 浏览器打开 http://localhost:5173（不要双击 index.html）
```

CHAPTER 07

创新点说明

7.1 技术创新点

#	创新点	具体实现	为什么是创新
T1	可解释的多 Agent 派发	派发理由 + 置信度实时写入时间线	常见 Agent 系统只做派发，不解释「为什么派给这个 Agent」
T2	连环追问机制	技术关键词触发 35% 概率连续深挖 2 轮	从「轮转」升级到「连续决策」，是 Agent 与机器人的分水岭
T3	BM25 + 端侧向量混合检索	参数调优 + Transformers.js + 融合权重 + 离线兜底	端侧语义检索的完整工程实现，不是调 API
T4	逐条展示两路得分的 RAG 引用	`bm25 0.82 · cos 0.61 · 融合 0.74`	检索过程完全透明，可当场核对
T5	可解释评分 + 证据链	每条分数引用对话原话 + 观察员标注	评分不是黑箱，每条都能追溯
T6	前后端评分引擎同构	Python 版与 JS 版算法一致，交叉验证	三级降级在评分层的具体实现
T7	MCP 工具运行时可验证	开发者模式内置 MCP 控制台	支持 MCP 的产品很多，能让评委当场验证调用链路的少
T8	代码沙盒入评分	Web Worker + 2s 熔断，通过率并入内容深度维度	把「编码能力」这个难量化的维度变成可量化信号

7.2 应用创新点

#	创新点	说明
A1	求职全流程闭环	简历匹配 → 简历优化 → 面试 → 报告 → 题库 → 求职看板，形成闭环而非单点工具
A2	题库 + 错题本 + 每日一练	把「背八股」变成可量化的学习过程
A3	职业探索匹配引擎	按专业 + 技能 + 兴趣匹配 24 个岗位，给出技能对比与学习路径
A4	AI 小助手（全局悬浮）	可拖动贴边冻结，检索知识库作答，使用引导一键直达
A5	开发者模式（面向评委）	把「技术透明度」做成一个面向评委的产品功能，而不是文档里的一段说明
A6	移动端体验优化	顶部导航、触控 ≥42px、输入框 16px 防 iOS 缩放、底部安全区适配

7.3 创新点总结

最大的创新不在单点技术，而在「可解释」这个设计哲学贯穿全产品

- Agent 派发可解释（为什么派给他）；
- RAG 检索可解释（两路得分怎么算）；
- 评分可解释（为什么是这个分）；
- 学习路径可解释（为什么练这个）；
- 降级可解释（为什么现在是 Mock）。

这种「全链路可解释」在 AI Native 应用里是稀缺的

——大多数产品追求「更智能」，但用户不知道智能从哪来。面镜反过来，每一步都留下可核查的证据。

CHAPTER 08

总结与展望

8.1 已完成

- **5 个 Agent 协作 + 动态派发 + 连环追问**的完整面试引擎；
- **BM25 + 端侧向量混合检索**的 RAG 层，含缓存、降级、可解释得分；
- **可解释评分 + 证据链 + 动态学习路径**的报告引擎；
- **三级降级**的完整链路（后端 → 直连 → Mock），演示不中断；
- **870 + 6 项自动化测试**，覆盖所有功能；
- **开发者模式**作为面向评委的技术透明度工具。

8.2 已知边界

边界	说明
演示版向量库	浏览器内 localStorage 单机索引（40 条），非分布式向量数据库
端侧模型	首次加载需下载 ~23MB；纯离线环境降级为 hashing 向量（仅词面）
MCP	本地模拟 Server，协议一致但非跨进程真实 MCP
代码沙盒	Web Worker 隔离，仅支持纯 JS 逻辑题
登录系统	演示级（本地混淆），非生产级认证
数据库	SQLite 单机，生产需替换 PostgreSQL

8.3 未来规划

方向	计划
RAG 升级	pgvector 持久化 + RRF 融合 + Cross-Encoder Rerank
Agent 升级	真实 MCP Server + 跨 Agent 工具发现 + 多轮规划
评分升级	LLM 评分 + Zod 约束 + 校准样本集量化一致性
多模态	Whisper ASR + 语速 / 停顿分析 + 白板代码识别
部署	多租户隔离 + 限流 + PostgreSQL + 对象存储
商业模式	院校就业中心合作 / 企业校招 SaaS / C 端订阅

8.4 致谢与声明

本项目开发过程中使用 AI 编程工具作为辅助手段，**CSS 主题样板、通用工具函数、测试用例骨架、部分页面文案由 AI 辅助生成**；**多 Agent 角色设计与派发策略、可解释评分规则与证据链、RAG 检索层参数调优与融合策略、业务架构与前后端契约、安全方案、测试体系均为团队独立设计与实现。**所有 AI 生成内容均经过团队逐行审查、修改与测试，完整 Git 提交记录可见演进过程。演示数据全部为人工合成示例，无爬取真实网络数据。

CHAPTER 10

团队信息与成员分工

面镜 FutureMirror 开发团队 • 福建农林大学桢队 · 技术文档 v1.0（对应产品 Beta 1.0）

角色	成员	GitHub
项目负责人 / 前端开发 / 后端开发	周鹏辉（PenguinZhou）	@Penguin-Zhou
文案与视觉美化	林宇豪（Vito）	—

10.1 项目负责人丨周鹏辉（PenguinZhou）

- 整体产品需求梳理、系统架构设计；完成多 Agent 角色划分、任务调度逻辑设计；
- 负责 Prompt 工程、打分评估规则、可解释评价体系、系统降级与容错方案设计；
- 统筹 Git 仓库搭建、规范代码提交，拆分细粒度 commit，保证开发历史完整可追溯；
- 承担前后端整体联调、功能测试，设计数据隐私安全与合规方案。

10.2 前端开发丨周鹏辉（PenguinZhou）

- 完成页面布局、组件开发（HTML/CSS/JS）；实现明暗双主题、流式对话输出、雷达图可视化、报告导出打印、弹窗交互；
- 开发本地存储、语音识别与语音朗读、开发者调试面板；完成离线 Mock 模式开发，对接后端接口并完成全流程调试。

> **备注**
> ：基础 UI 代码由 AI 编程工具辅助生成，本人独立完成业务逻辑编写、代码审核、逻辑纠错与 BUG 修复。

10.3 后端开发丨周鹏辉（PenguinZhou）

- 使用 FastAPI 搭建代理服务，实现 LLM 流式 SSE 消息转发，解决浏览器跨域问题；
- 完成 API 密钥安全托管、请求超时捕获、异常信息处理；搭建 SQLite 数据持久化、简易 RAG 检索模块；
- 编写项目部署脚本、依赖环境清单，验证 10 分钟本地一键复现流程。

10.4 文案与视觉美化丨林宇豪（Vito）

- 撰写 README 文档、AI 辅助开发声明、全套技术说明书与赛事申报材料；
- 撰写演示视频脚本，配合完成内容准备；
- 负责全部技术文档排版美化、图文规整，优化文档视觉观感。

本技术文档与仓库 README 为同一套交付材料：README 含「项目介绍 / 技术架构 / 安装运行 / 团队信息 / 版本更新日志」；技术文档为 11 章完整技术说明，二者互相指向。

CHAPTER 11

版本更新日志（摘要）

本日志为团队真实开发进程的版本记录

Git 提交历史自

Beta 版本

起建立，

仅包含 BETA 1.0 及之后更新

；v1–v22 演进以本摘要及仓库 README「附：版本更新日志」完整留存，未在 Git 中逐条提交。完整逐版本明细（新增/优化/修复/数据指标）见 HTML 版技术文档与 README。

版本	日期	核心变更
v23	待排期	规划中 · 完整版对齐 · 生产级 RAG（pgvector/RRF/Cross-Encoder）· 多模态扩展（Whisper/白板识别）
Beta 1.0	2026.10	检索层升级（BM25+端侧向量混合检索 · 权重同源自托管）与版本定位，当前发布版本
v22	2026.10	后端适配与工程化：FastAPI 落地、三级降级、向量索引状态条
v21	2026.10	悬浮球拖拽冻结 + 移动端美化
v20	2026.10	题库增强 + AI 小助手（全局悬浮） + 移动端导航顶部化
v19	2026.10	题库小测模块 + VIP 会员体系
v18	2026.10	产品闭环深化：求职管理看板 + 面试真实感（答题计时/连环追问/语音反馈）
v17	2026.10	深海蓝 AI Native 视觉重构（Arkira 风格）
v16	2026.10	个人中心：头像系统 · 偏好与数据管理
v15	2026.10	Beta · 工作台精简与互动升级
v14	2026.10	开发者模式升级（加密视图/环境检测/诊断导出）
v13	2026.09	评审专区（加密 + 一键进入）
v12	2026.09	可验证运行时能力三件套：MCP 控制台 / 代码沙盒 / 本地 PII 脱敏
v11	2026.09	目标岗位下拉选择器
v10	2026.09	导航重构（左栏+底部 Tab）与数据扩充（岗位 8→24 / 面经 8→40）
v9	2026.08	修复 HTML 结构 + 细节完善
v8	2026.08	多页面路由 + 导航系统
v7	2026.08	首页落地页 + 游客模式
v6	2026.08	真实模型修复 + 新板块 + UI 重构
v5	2026.08	登录系统 + UI 全面升级
v4	2026.08	深度完善版
v3	2026.08	深度完善 · 修复模型调用
v2	2026.08	体验完善版
v1	2026.08	初始原型